HashDit

Smart Contract Security

ListaDao LiquidationVault

Security Audit Report

ListaDao

08 Jul 2026 - 09 Jul 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	08 Jul 2026 — 09 Jul 2026
Overall Risk	**High**
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/207
Commit	feature/liquidation-vault

Smart Contract List

No.	Contract Name	Verdict	Details
1	LiquidationVault.sol	Medium	See Findings
2	Liquidator.sol	High	See Findings
3	BrokerLiquidator.sol	High	See Findings
4	deploy_liquidationVault.sol	Medium	See Findings
5	deploy_liquidationVaultConfig.sol	Medium	See Findings

Findings

[H01] BOT can bypass the LiquidationVault loss guard through direct sellToken and sellBNB on the liquidators	
Contract	Liquidator.sol & BrokerLiquidator.sol
Severity	High
Description	**Description** The LiquidationVault enforces a swap loss guard through _guardSwapLoss that caps both per swap loss and cumulative daily loss when inventory is sold. However, the Liquidator and BrokerLiquidator contracts expose their own non-pausable sellToken and sellBNB functions, callable by the BOT role, that execute swaps directly against a whitelisted pair without routing through the vault or its loss guard. **Impact** In these functions the BOT alone supplies amountOutMin, so the BOT fully controls the accepted slippage on each sale. A compromised or malicious BOT can therefore sell tokens at an arbitrary loss through the liquidator level sellToken and sellBNB paths, completely sidestepping the per swap and per day loss limits that the vault was designed to enforce. 1. This exposure is most significant before the shared LiquidationVault is set up, meaning before deployment completes and fundSource is set on the liquidators. During that window the vault based protection is not in effect at all, yet the BOT can already move funds out through these direct sale functions while still choosing its own slippage. 2. The problem is made worse by the absence of a deployment script that atomically sets fundSource and migrates all pre-existing liquidator balances into the vault. The provided configuration script only wires setFundSource and leaves reserve migration, through collectERC20 and collectETH, as a separate manual step that it recommends be executed atomically through a multisig rather than enforcing it in code. As a result, there is a possible window in which funds continue to sit inside the Liquidator and BrokerLiquidator contracts, still reachable by the BOT controlled sellToken and sellBNB functions with BOT chosen slippage and still outside the loss guard. 3. Lastly, given the fundSource can be reset to the zero address via the setFundSource within the Liquidator and BrokerLiquidator, it can immediately allow the BOT role to once again perform liquidations → malicious token sales

	without going through a loss guard once again, causing losses to ListaDao from pre-funded tokens for liquidations
Recommendation	Consider the following: • Routing all BOT initiated sales through the same oracle based loss guard used by the LiquidationVault, so that sellToken and sellBNB on the Liquidator and BrokerLiquidator are subject to the identical per swap and per day loss caps rather than relying on a BOT supplied amountOutMin. • During deployment, ensure the fund source and loss guard are fully configured before the BOT role is able to execute any sales, so that no window exists in which BOT controlled funds remain outside the guard.
Status	Acknowledged, ListaDao has mentoned that the Liquidator/BrokerLiquidator will no longer hold funds after the migration to the LiqudiationVault. However, this issue is maintained as the code logic within the setFundSource indicates a potential of migrating the LiquidationVault back to the Liquidator/BrokerLiquidator itself, as well as a larger maxDailyLossUsd allowed.

[M01] Missing rate cap on provideFund() lets pooled loanToken be depleted into collateral inventory	
Contract	LiquidationVault.sol, BrokerLiquidator.sol & Liquidator.sol
Severity	Medium
Description	**Description** In both Liquidator and BrokerLiquidator, the onMoolahLiquidate() callback contains a pre funded branch, else if (fundSource != address(0)), that pulls the exact shortfall repaidAssets - bal from the shared LiquidationVault through provideFund(). This branch is reached by liquidate() and liquidateSmartCollateral(), both restricted to onlyRole(BOT). The two swap branches, for swapCollateral and swapSmartCollateral, enforce if (out < repaidAssets) revert NoProfit(), but the pre funded branch performs no profitability check, and provideFund() has no per transaction or per day cap. The pre funded branch is identical in both contracts, so the issue applies equally to each.

JavaScript

```
   } else if (fundSource != address(0)) {
      // normal pre-funded path (no self-funded swap). Pull the exact
shortfall from the vault.
      // repaidAssets is already exact here (Moolah computed it before
this callback, before its
      // transferFrom). Local balance is consumed first, so the vault
only tops up the difference.
      uint256 bal = arb.loanToken.balanceOf(address(this));
      if (bal < repaidAssets) {
```

```
        ILiquidationVault(fundSource).provideFund(arb.loanToken,
repaidAssets - bal);
      }
    }
```

Additionally, Liquidator and BrokerLiquidator each held their own loanToken reserves and collateral inventory, so a problem confined to one did not affect the other. The change registers both against a single LiquidationVault and points both fundSource values at it. Both now pull repayment shortfalls from the same pool through provideFund inside onMoolahLiquidate, and both reflow inventory into the same pool, allowing high liquidation volume of each liquidator contract to potentially simultaneously affect the other.

Impact

This does not allow a direct drain of the vault. Moolah's liquidation incentive guarantees the seized collateral is worth >= repaidAssets at the oracle price, because repaidAssets <= seizedAssets * oraclePrice when liquidationIncentiveFactor >= 1.0, and any bad debt is socialized to suppliers rather than charged to the liquidator. The vault therefore always receives collateral worth at least what it paid, valued at the oracle price.

The real risk is a liveness and inventory concern. Every non swap liquidation converts the vault's shared loanToken reserves into collateral inventory. With no rate cap, a compromised or buggy BOT, or simply a burst of liquidations that outpaces the vault's sell and collect operations, can deplete the pooled loanToken until provideFund() reverts on insufficient balance. At that point every fund source backed liquidation across all registered liquidators will be DoSed.

Recommendation	Consider adding a configurable, per liquidator ceiling on provideFund, applied per transaction and per day, so activity on one liquidator cannot draw the shared pool below what the other needs and so a compromised BOT or high liquidation volume cannot drain the LiquidationVault on demand.
Status	Acknowledged, no fix implemented. It is presumed that ListaDao admins will monitor and manage sufficient liquidity within the LiquidationVault appropriately.

[M02] Global maxSwapLossBp is too lenient for peg and LST pairs and should be a per path cap	
Contract	LiquidationVault.sol
Severity	Medium
Description	**Description** The LiquidationVault loss guard applies a single global maxSwapLossBp, defaulting to 50_000 in parts per million, which is 5%, to every swap regardless of the token pair being traded. The check compares oracle USD values as valueOut * 1e6 >= valueIn *

	(1e6 - maxSwapLossBp), and the oracle already prices each leg correctly, including the exchange rate of liquid staking tokens such as slisBNB.
	
	JavaScript
	`// per-swap loss cap`
	`require(valueOut * BPS_DENOM >= valueIn * (BPS_DENOM - maxSwapLossBp), OracleLoss());`
	
	For pairs that trade near a known ratio, a fair swap returns valueOut approximately equal to valueIn minus only the DEX fee and a small amount of book slippage. Expected fair loss on these pairs is well under 1%, for example roughly 0.05% to 0.3% for slisBNB to BNB, roughly 0.01% to 0.1% for lisUSD to a stablecoin, and roughly 0.1% to 0.5% for BTCB to solvBTC. Against those expectations, a 5% allowance is 10 to 500 times looser than a realistic slippage.
	
	Impact
	On a peg or LST pair, 5% is far above normal slippage that the per-swap maxSwapLossBp global guard catches almost nothing on exactly the pairs where any deviation is the clearest signal that something is wrong. A 5% allowance is only defensible for volatile collateral being sold into a stable token, where genuine DEX slippage on a large seizure can approach that level. A single global value cannot serve both cases at once. As such, the BOT role or external users can sandwich such swaps to cause MEV losses when looser slippage is utilized.
	
	The per UTC day cumulative cap, defaulting to 1000e8, which is 1000 USD, limits total daily loss and prevents this from draining the vault, so the issue is limited to a leakage capped at the daily loss limit.. Even so, a single stable pair swap of about 20,000 USD at the full 5% allowance consumes the entire daily budget of 1000 USD in one transaction on a trade that should have lost close to nothing, and the leak compounds over time if repeatedly exercised.

Recommendation	Consider the following: - Introduce an admin only function that allow setting bi-directional per path swapLossBp mapped to the input and output token, falling back to the global maxSwapLossBp when a path is not configured. as a conservative value for unconfigured, typically volatile, pairs - In _guardSwapLoss, resolve the effective cap as the path specific value when set and the global value otherwise, then apply the existing comparison with that resolved cap.
Status	Acknowledged, no fix implemented, ListaDao has indicated that the daily loss incurred is acceptable and admins will tune maxSwapLossBp where necessary.

[M03] Flash liquidation swap path bypasses the LiquidationVault loss guard and daily loss cap	
Contract	BrokerLiquidator.sol & Liquidator.sol
Severity	Medium
Description	**Description** The LiquidationVault protects value conversions with _guardSwapLoss, which enforces a per swap cap of maxSwapLossBp (default 5%) valued at the oracle price and a per day cumulative cap of maxDailyLossUsd (default $1000) tracked in dailyLossAccum. These protections exist only in the vault's sellToken and sellBNB. For any unhealthy position in a whitelisted market, the BOT can choose between three interchangeable routes on the Liquidator and BrokerLiquidator, and only one of them is protected. 1. The first route, liquidate and liquidateSmartCollateral, sets both the swapCollateral and swapSmartCollateral flags to false, so the callback pre funds the repayment from the vault, seizes the collateral, and reflows it to the vault as inventory. That inventory is later converted through the guarded vault.sellToken and vault.sellBNB, so the loss guard and the daily cap apply. 2. The second route, flashLiquidate and flashLiquidateSmartCollateral, sets the swap flag, so the seized collateral is swapped to loanToken inside onMoolahLiquidate through a low level pair.call. This swap runs inside the liquidator, which holds no oracle reference and never updates the vault's dailyLossAccum. The only economic check on this path is out >= repaidAssets, which confirms the swap covers the debt but does not confirm that the swap captured the collateral's full value. Because Moolah's liquidation incentive guarantees the seized collateral is worth more than or equal to the repaidAssets, a swap that returns exactly repaidAssets passes while leaking the entire liquidation bonus to the BOT role/counter party. 3. The third route is a modified version of the second route, where the BOT role can utilize flashLiquidate and flashLiquidateSmartCollateral to perform regular liquidation by atomically transferring loan/collateral tokens to self-fund liquidations. The same impact occurs as per route 2. **Impact** The result is that the vault's loss guard and daily loss cap are optional as a compromised or malicious BOT role can leak value by routing every conversion through flashLiquidate, where there is no per swap oracle cap and no aggregate ceiling, so it can extract the bonus on every liquidation, every day, with no bound. In the best case it returns immediate full profit with no inventory risk, and in the worst case it leaks the full liquidation bonus with no cap,

	One offsetting property is that the flash route is self funded and never calls provideFund, so it keeps functioning when the vault is paused or drained. This only helps for profitable positions, because when the collateral has fallen below the debt the flash route reverts on NoProfit, and only the pre funded route can clear those positions. Since pausing or draining the vault is what disables the pre funded route, the bad debt positions that matter most during a market crash still cannot be liquidated.
Recommendation	Consider adding a fair value guard to the flash liquidation callback swap that bounds out below by the oracle value of the seized collateral minus a tolerance, mirroring _guardSwapLoss, and accrue the realized loss into a shared daily accumulator so the per swap cap and the per day cap cannot be bypassed by choosing the flash route by the BOT role
Status	Acknowledged, no fix implemented, ListaDao has accepted this as a design decision wherein the only affected funds are the leaked liquidation bonuses.

[M04] Incomplete token whitelist for loan tokens and collateral tokens within the vault	
Contract	deploy_liquidationVaultConfig.sol
Severity	Medium

Description	**Description**
	deploy_liquidationVaultConfig.sol whitelists only WBNB, BTCB, slisBNB, and solvBTC, which are all collateral side tokens. The markets that the liquidators serve use loanTokens such as lisUSD , USDT and USD1, none of which are whitelisted. Both sellToken and sellBNB require tokenWhitelist[tokenOut], and the core purpose of the vault is to sell seized collateral into the loanToken.
	
	JavaScript
	`// 1) Oracle + sell whitelist (token + pair). BNB_ADDRESS must be whitelisted for sellBNB/collectETH.`
	`vault.setOracle(resilientOracle);`
	`vault.setTokenWhitelist(BNB_ADDRESS, true);`
	`vault.setTokenWhitelist(WBNB, true);`
	`vault.setTokenWhitelist(BTCB, true);`
	`vault.setTokenWhitelist(slisBNB, true);`
	`vault.setTokenWhitelist(solvBTC, true);`
	`vault.setPairWhitelist(pair, true);`
	
	Impact
	With no loanToken whitelisted, a sale of collateral into the loanToken reverts with NotWhitelisted, so the vault can accumulate collateral but cannot replenish the loanToken it needs for provideFund. This breaks the refill loop and worsens the shared

	pool starvation risk. BOT collection of loanToken reserves through collectERC20 also requires the token to be whitelisted, so the BOT cannot sweep loanToken either. Additionally, The vault sells whitelist lists WBNB, BTCB, slisBNB, and solvBTC, but the liquidators may be whitelisted for markets whose collateral is not in that set, such as LP, PT, or other collateral tokens. When such a market is liquidated, the seized collateral is reflowed to the vault and cannot be sold, because it is not in tokenWhitelist, has no whitelisted pair, or is not priceable by the oracle. This strands loanToken value in inventory the vault cannot reconvert. The comment instructs the operator to copy the live liquidator whitelist set, but the hardcoded example is incomplete.
Recommendation	Add required loanToken/collateralToken used by any whitelisted market to tokenWhitelist before the vault is relied upon, so that sellToken and sellBNB can produce loanToken for liquidation and the loanToken reserve can be collected appropriately from the Liquidator contracts for migration.
Status	Fixed, now whitelists all underlying loan and collateral tokens that is allowed to be swapped within the LiquidationVault.

[M05] Config script targets are hardcoded to address(0)	
Contract	deploy_liquidationVaultConfig.sol
Severity	Medium

Description	**Description**
	deploy_liquidationVaultConfig.sol declares vault, liquidator, and brokerLiquidator as address(0) with a comment to fill in the deployed addresses before running.
	
	JavaScript
	`/// Fill in the deployed addresses before running.`
	`contract LiquidationVaultConfigDeploy is DeployBase {`
	`  LiquidationVault vault = LiquidationVault(payable(address(0)));`
	`  Liquidator liquidator = Liquidator(payable(address(0)));`
	`  BrokerLiquidator brokerLiquidator =`
	`BrokerLiquidator(payable(address(0)));`
	
	Impact
	The configuration calls within the deployment script are hence void external calls, and a call to an zero address with no code would revert, so running the script without editing the addresses beforehand will fail, rendering the deployment script non-usable.

Recommendation	Ensure that the required addresses are filled before running the required deploy_liquidationVaultConfig deployment script, considering these addresses are already live and deployed proxies with a constant address.
Status	Fixed, now enforces zero-addresses when performing the atomic deploy_liquidationVaultConfig.sol, deferring input checks of accurate live and deployed addresses to the deployer.

[M06] setFundSource on the live liquidators cannot be executed by the deployer in a single broadcast	
Contract	deploy_liquidationVaultConfig.sol
Severity	Medium

Description	**Description**
	Steps 1, 2, and 4 of the config script act on the freshly deployed vault where the deployer is MANAGER, so they succeed. Step 3 calls setFundSource on the already live Liquidator and BrokerLiquidator, which is restricted to their MANAGER role. On mainnet that role is held by the production 3/6 multisig , not the deployer, so a single vm.startBroadcast(deployer) execution reverts at step 3.
	
	JavaScript
	`// 3) Point each liquidator at the vault (grey-scale cut-over; 0 rolls back to legacy behavior).`
	`liquidator.setFundSource(address(vault));`
	`brokerLiquidator.setFundSource(address(vault));`
	
	Impact
	The script thus assumes that the deploy time authority over the new LiquidationVault with production authority over the existing liquidators, and the comment acknowledging that this should be executed atomically through a multisig contradicts the plain broadcast structure of the script.

Recommendation	Consider splitting the above flow by letting the deployer configure the vault, and have a separate multisig batch register the liquidators in the vault and call setFundSource atomically
Status	Fixed, with the following caveats: • The wire script must be a Safe batch in one atomic multisig transaction. It cannot run as a deployer broadcast, given after transferRole the deployer holds no roles, and liquidator.setFundSource requires MANAGER on the live liquidators (the production Safe), which the deployer never holds.

- The wire script migrates only loan tokens, given _loanTokens() covers WBNB/lisUSD/USDT/USD1 but not existing collateral inventory (BTCB/slisBNB/solvBTC) sitting in the live liquidators. Those are left behind at cutover and rely on later reflow/collect.

[M07] RevenueCollector BOT can drain the entire LiquidationVault and halt all pre funded liquidations	
Contract	LiquidationVault.sol
Severity	Medium

Description	**Description**
	The LiquidationVault intentionally reuses the ILiquidator withdraw selectors, withdrawERC20() and withdrawETH(), so that the unmodified RevenueCollector can pull fees from it. Once the vault is wired as the collector through vault.setRevenueCollector(RC) and registered in the collector's liquidator set through RC.updateLiquidator(vault, true), the RevenueCollector BOT role can call claimLiquidationFee() or claimLiquidationFees().
	
	The vault gate _requireManagerOrRevenueCollector() then passes because msg.sender equals vault.revenueCollector. The collector only checks asset != address(0) and amount > 0, so any token and any amount can be specified with no per transaction or per day cap. A compromised RevenueCollector BOT can therefore pull the vault's entire balance of any token into the collector.
	
	JavaScript
	`/// @dev Withdraw an ERC20 to msg.sender. Gate: MANAGER (wind-down/rebalance) or the registered`
	`///      revenueCollector (fee pull). Same selector as ILiquidator so RevenueCollector can call it.`
	`function withdrawERC20(address token, uint256 amount) external {`
	`_requireManagerOrRevenueCollector();`
	`token.safeTransfer(msg.sender, amount);`
	`emit Withdraw(msg.sender, token, amount);`
	`}`
	
	`/// @dev Withdraw native BNB to msg.sender. Gate as withdrawERC20.`
	`function withdrawETH(uint256 amount) external {`
	`_requireManagerOrRevenueCollector();`
	`msg.sender.safeTransferETH(amount);`
	`emit Withdraw(msg.sender, BNB_ADDRESS, amount);`
	`}`
	
	Impact
	1. Because the vault now holds all pooled liquidation reserves, draining its loanToken breaks the next pre funded liquidation. onMoolahLiquidate() calls provideFund(loanToken, repaidAssets - bal), the vault's transfer reverts on

	insufficient balance, and the whole liquidate() or liquidateSmartCollateral() reverts. This halts every fund source backed liquidation across both Liquidator and BrokerLiquidator. During volatile markets, positions that cannot be liquidated promptly can accrue bad debt that is socialized across all lenders. 2. Additionally, given collectERC20/collectETH allows the BOT role to consolidate all preexisting funds within the Liquidator/BrokerLiquidator contracts, they can immediately and repeatedly call collectERC20/collectETH → withdrawERC20/withdrawETH to repeatedly cause the DoS. 3. The vault's withdrawERC20() and withdrawETH() are not whenNotPaused, so pausing the vault does not close this path and the PAUSER cannot defend against an in progress drain. Note that this is purely a DoS vector as the only way funds leave the RevenueCollector is emergencyWithdraw(), which is restricted to the MANAGER role, so pulled funds remain MANAGER recoverable. However, it would force the MANAGER role to prefund the LiquidationVault again in which the griefing attack can be repeated.
Recommendation	Consider only allowing the MANAGER to perform withdrawERC20 and withdrawETH calls.
Status	Acknowledged, ListaDao has mentoned that the Liquidator/BrokerLiquidator BOT addresses are distinct from the RevenueCollector BOT addresses, so the RevenueCollector BOT addresses are trusted to not allow this to occur. It is presumed that sufficient off-chain private key protection is in place to prevent a malicious/compromised BOT EOA on the RevenueCollector side (excluding multi-sig) from performing such a DoS.

[L01] Fail-open in _oracleValue when a swap leg rounds down to zero disables the loss guard	
Contract	LiquidationVault.sol
Severity	Low
Description	**Description** The LiquidationVault._oracleValue() function computes the 8-decimal USD value of a token amount using integer division. When the result truncates to 0, the function returns 0 with no check. _guardSwapLoss() depends on this value for both of its protections, and a zero return can silently disable them. JavaScript `/// @dev 8-decimal USD value of `amount` of `token`. `dec = 18` for native BNB. peek==0 => revert.`

```
  function _oracleValue(address token, uint256 amount) private view
returns (uint256) {
    uint256 price = IOracle(oracle).peek(token); // 8-decimal USD
    require(price > 0, OracleZero());
    uint256 dec = token == BNB_ADDRESS ? 18 :
IERC20Metadata(token).decimals();
    return (amount * price) / (10 ** dec);
  }
```

The impact splits into three cases depending on which leg rounds to 0:

1. valueOut == 0 while valueIn > 0: the per-swap check valueOut * 1e6 >= valueIn * (1e6 - maxSwapLossBp) becomes 0 >= valueIn * (1e6 - maxSwapLossBp), which is false, so the call reverts with OracleLoss. This case is safe and fails closed on its own.

2. valueIn == 0 (regardless of valueOut): the per-swap check becomes valueOut * 1e6 >= 0, which is always true, so the swap passes no matter how little tokenOut comes back. The daily loss accounting also records 0, because loss = valueIn - valueOut cannot be positive when valueIn = 0. The guard is fully bypassed for that swap.

3. valueIn == 0 and valueOut == 0: the check reduces to 0 >= 0, which passes, and again loss = 0 is recorded. Both the per-swap cap and the daily cumulative cap are disabled for that swap.

The dangerous cases are 2 and 3, where the input value is 0. In those cases the vault can send out tokenIn and receive almost nothing in return while _guardSwapLoss() enforces nothing and the daily loss accumulator stays flat.

Impact

With the tokens currently in scope this is not exploitable, since every whitelisted token has a high value per unit and no realistic trade size rounds to 0, so the amount escaping the guard is only dust worth less than 1e-8 USD. There is an exception where a very small amount is utilized by the BOT role to perform the swap over many transactions, but that would take a large and unrealistic amount transactions just to bypass the loss gap.

The gap is latent. It becomes materially reachable if a future whitelist addition uses high decimals combined with a sub-cent price, or if the oracle returns an unexpectedly small price, because then a normally sized tokenIn amount carrying real economic

	value can round to valueIn = 0, leave the vault for little or nothing, and still pass the guard.
Recommendation	Consider • Adding a check within _oracleValue() fail closed by requiring the computed value to be greater than 0 before returning it, for example require(value > 0) with a dedicated error. This forces any swap leg whose value truncates to 0 to revert instead of bypassing _guardSwapLoss(), and it covers both the valueIn = 0 and the valueIn = 0, valueOut = 0 cases, keeping behavior consistent with the existing revert on a zero oracle price. • Additionally,, only whitelist tokens where 10 ** dec / price is small enough that realistic trade sizes never round to 0.
Status	Fixed, now ensures value computed within the _oracleValue is greater than zero.

[L02] Risk parameters are left at defaults that are too small for a pooled reserve	
Contract	deploy_liquidationVaultConfig.sol
Severity	Low
Description	**Description** Depending and in conjunction with the fixes towards [M02] and [I03], the config script does not set maxSwapLossBp or maxDailyLossUsd, so the vault keeps the defaults from initialize, which are 50_000 meaning 5% per swap and 1000e8 meaning $1000 cumulative per UTC day. **Impact** For a contract intended to hold all protocol liquidation inventory, a $1000 daily cumulative loss budget could be too small. It makes the daily cap easy to exhaust, which blocks all further loss incurring sales for the rest of the day, and it acts as an absolute ceiling that a single legitimate large sale can exceed and revert on.
Recommendation	Consider implementing the fixes in [M02] and [I03] and refactoring the deploy_liquidationVaultConfig to assign appropriate maxSwapLossBp and maxDailyLossUsd per directional pair appropriately. This can also be performed in a separate script.
Status	Fixed, maxDailyLossUsd raised to $100,000, along with the addition of the resetDailyLoss function to allow MANAGER to reset daily loss during large sale volumes due to high liquidation activity.

[I01] Missing zero address check in setTokenWhitelist	
Contract	LiquidationVault.sol

Severity	Informational

Description	**Description**
	The LiquidationVault.setTokenWhitelist() function updates the token whitelist without validating that the provided token address is non zero. This differs from other configuration setters in the contract, such as the pair whitelist setter, which reject the zero address.
	
	JavaScript
	`function setTokenWhitelist(address token, bool status) external onlyRole(MANAGER) {`
	`  require(tokenWhitelist[token] != status, WhitelistSameStatus());`
	`  tokenWhitelist[token] = status;`
	`  emit TokenWhitelistChanged(token, status);`
	`}`
	
	Impact
	As a result, the zero address can be added to the token whitelist. While this does not create a direct exploit on its own, it allows an invalid entry into a security relevant allow list and weakens input hygiene, which can lead to confusing state or mask configuration mistakes.

Recommendation	Add a zero address check to setTokenWhitelist() so that it reverts when the token address equals the zero address, matching the validation already applied in the other whitelist and address setters in the contract.
Status	Fixed as recommended, now adds a zero address check so that it reverts with the ZERO_ADDRESS string when invoked.

[I02] Setter and admin functions do not emit the previous configuration value	
Contract	LiquidationVault.sol
Severity	Informational
Description	**Description** The administrative setter functions in LiquidationVault emit events for their updates but do not include the previous value alongside the new value. This applies across the full set of configuration setters, including setLiquidator(), setTokenWhitelist(), setPairWhitelist(), setOracle(), setMaxSwapLossBp(), setMaxDailyLossUsd(), and setRevenueCollector(). Each of these changes a security relevant parameter, such as the liquidator registry, the token and pair whitelists, the oracle address, the per swap loss cap, the daily loss cap, and the revenue collector, yet the emitted events record only the resulting state. **Impact**

	Because the prior value is never included in the logs, off chain monitoring tools, indexers, and auditors cannot reconstruct the complete history of configuration changes from events alone. They can observe the new value but must separately query the historical state to learn what it replaced. This reduces transparency and makes it harder to detect, trace, or review unexpected or unauthorized parameter changes across the contract's administrative surface.
Recommendation	Consider updating the events emitted by every setter and admin function in the contract to include both the old value and the new value.
Status	Acknowledged, no fix implemented, ListaDao has indicated that new-value events plus historical state queries are sufficient for their indexing.

[I03] Shared daily loss accumulator lets one lossy swap block all further sells until the UTC reset	
Contract	LiquidationVault.sol
Severity	Informational
Description	**Description** The LiquidationVault loss guard in _guardSwapLoss() tracks losses from sellToken() and sellBNB() using a single global accumulator, dailyLossAccum, that is shared across every whitelisted pair and token, and profit is never netted against it. Once dailyLossAccum reaches maxDailyLossUsd, every subsequent swap that produces any loss > 0 reverts with DailyLossExceeded, regardless of which pair is involved, until the next UTC day resets the accumulator.

JavaScript

```
 /// @dev Oracle loss guard. Uses balance-diff actual amounts.
Fail-closed on a zero price.
  ///      (1) per-swap cap: valueOut·1e6 >= valueIn·(1e6 -
maxSwapLossBp);
  ///      (2) per-UTC-day cumulative loss cap (profit does not offset).
  function _guardSwapLoss(address tokenIn, uint256 amountIn, address
tokenOut, uint256 amountOut) private {
    uint256 valueIn = _oracleValue(tokenIn, amountIn);
    uint256 valueOut = _oracleValue(tokenOut, amountOut);

    // per-swap loss cap
    require(valueOut * BPS_DENOM >= valueIn * (BPS_DENOM -
maxSwapLossBp), OracleLoss());

    // per-UTC-day cumulative loss cap (profit is not credited)
    uint256 today = block.timestamp / 1 days;
    if (today != dailyLossDay) {
      dailyLossDay = today;
      dailyLossAccum = 0;
    }
    uint256 loss = valueIn > valueOut ? valueIn - valueOut : 0;
    if (loss > 0) {
```

```
        uint256 accum = dailyLossAccum + loss;
        require(accum <= maxDailyLossUsd, DailyLossExceeded());
        dailyLossAccum = accum;
        emit DailyLossAccrued(today, loss, accum);
      }
    }
```

Impact

This gives the BOT role a possible denial of service. A compromised BOT can sell roughly $20k of inventory at the maximum permitted 5% slippage, which produces a loss of 1000e8 and reaches the default maxDailyLossUsd of 1000e8 in a single swap. From that point the entire loss incurring sell path is frozen for the rest of the day across all pairs, and only zero loss or profitable swaps still succeed..

The same limit also harms honest operation. Because maxDailyLossUsd is an absolute figure and profit does not offset accrued loss, it effectively caps the absolute loss of a single swap at $1000. For a contract designed to hold all protocol liquidation inventory, one large legitimate inventory sale can exceed $1000 of slippage and revert outright, with no compromise required.

Recommendation

Consider the following:

1. Replace the single global cap with per pair configuration set by MANAGER. Configure a per pair maximum slippage for the per swap check and a per pair daily loss budget for the cumulative check, keyed on the pair or the tokenIn and tokenOut combination.
2. As an alternative or complementary mitigation, add an admin only function, gated by the MANAGER role, that resets dailyLossAccum to 0 and optionally sets dailyLossDay to the current day. This lets the operator restore the sell path within the same UTC day after a legitimate large sale or a false trip, without waiting for the midnight reset.
3. Lastly, ListaDao can monitor the swap volume and adjust the maxDailyLossUsd via setMaxDailyLossUsd if required

Status

Fixed, maxDailyLossUsd raised to $100,000, along with the addition of the resetDailyLoss function to allow MANAGER to reset daily loss during large sale volumes due to high liquidation activity.

	The only caveat is that the MANAGER can now reset daily limits whenever they like, so a compromised MANAGER role can serve as both a malicious BOT role and MANAGER to potentially bypass loss guards. This would require a compromised MANAGER role.

[I04] Missing event emission in initialize	
Contract	LiquidationVault.sol
Severity	Informational
Description	**Description** The LiquidationVault.initialize() function sets up the initial maxSwapLossBp value of 50_000 and the initial maxDailyLossUsd value of 1000e8, without emitting the MaxSwapLossBpChanged and MaxDailyLossUsdChanged respectively JavaScript `function initialize(address admin, address manager, address pauser, address bot) public initializer { require(admin != address(0), ZERO_ADDRESS); require(manager != address(0), ZERO_ADDRESS); require(pauser != address(0), ZERO_ADDRESS); require(bot != address(0), ZERO_ADDRESS); __AccessControl_init(); __Pausable_init(); __ReentrancyGuard_init(); _grantRole(DEFAULT_ADMIN_ROLE, admin); _grantRole(MANAGER, manager); _grantRole(PAUSER, pauser); _grantRole(BOT, bot); _setRoleAdmin(BOT, MANAGER); maxSwapLossBp = 50_000; // 5% maxDailyLossUsd = 1000e8; // 1000 USD (8-decimal) }` **Impact** Since initialization defines the starting security relevant state of the contract, the absence of an event means off chain monitoring tools, indexers, and auditors cannot observe the initial configuration directly from logs. This weakens transparency at the exact moment the contract's parameters and roles are established.
Recommendation	Consider emitting the appropriate events within initialize() for the initial maxSwapLossBp and maxDailyLossUsd values.
Status	Fixed, now emits the MaxSwapLossBpChanged and MaxDailyLossUsdChanged event within the initialize function.

[I05] Liquidator plain liquidate and flashLiquidate lack the smart collateral guard present in BrokerLiquidator	
Contract	Liquidator.sol
Severity	Informational
Description	**Description** The Liquidator contract's plain liquidate() and flashLiquidate() functions do not verify whether the market's collateral token is a SmartProvider StableSwapLPCollateral token. This differs from BrokerLiquidator.liquidate(), which reverts with SmartCollateralMustUseDedicatedFunction through its _isSmartCollateral guard. As a result, a BOT can point the plain liquidate() or flashLiquidate() at a smart collateral market, seize the StableSwapLPCollateral LP token, and reach _reflow(params.collateralToken), which tries to push the seized LP token into the LiquidationVault. The vault cannot redeem StableSwapLPCollateral, because only the SmartProvider, which is the token minter, can burn it, and the vault exposes no redeem entrypoint. **Impact** Currently, a fund lock is not possible, but only because of an unrelated mechanism rather than an intentional check. StableSwapLPCollateral restricts transfer and transferFrom to MOOLAH or holders of the TRANSFERER role through the onlyMoolahOrTransferer modifier, given the Liquidator contract is not expected to hold this role However, in the event a similar configuration (e.g. LendingBroker + SmartProvider) market is routed through the Liquidator contract, such an issue could cause fund lock in the future.
Recommendation	Consider adding an explicit smart collateral check to Liquidator.liquidate() and flashLiquidate(), mirroring the _isSmartCollateral guard in BrokerLiquidator, If not, ensure that the Liquidator.sol contract is never granted the StableSwapLPCollateral TRANSFERER role.
Status	Acknowledged, no fix implemented. ListaDao has inidicated that sufficient configuration steps to blacklist token flow into LiquidationVault will be taken. It is noted that a reflow token whitelist could be implemented within Liquidator/BrokerLiquidator, but this is not present in the code logic for now. This is still safe currently as the Liquidator contract will not hold the StableSwapLPCollateral TRANSFERER role.

[I06] setMaxDailyLossUsd lacks zero and bounds validation, which can block swaps

Contract	Liquidator.sol
Severity	Informational

Description	**Description**
	The LiquidationVault.setMaxDailyLossUsd() function assigns the new daily loss limit without validating the input for a zero value or for reasonable bounds. The daily loss limit is enforced in the swap loss guard, where the accumulated loss for the day must stay less than or equal to maxDailyLossUsd.
	
	JavaScript
	`function setMaxDailyLossUsd(uint256 usd8) external onlyRole(MANAGER) {`
	`  maxDailyLossUsd = usd8;`
	`  emit MaxDailyLossUsdChanged(usd8);`
	`}`
	
	Impact
	• If a MANAGER sets this value to 0, then any swap that produces even a minimal loss will push the accumulated loss above the limit and cause the guard to revert. This effectively blocks all loss making sells, leaving only exactly break even or profitable swaps able to execute, which can stall legitimate liquidation activity.
	• The absence of an upper bound also allows the value to be set arbitrarily high, which silently weakens or removes the intended cumulative loss protection. Because there is no sanity check on the input, a misconfiguration can either freeze normal selling or disable the safeguard.

Recommendation	Consider adding input validation to setMaxDailyLossUsd() that rejects a zero value and enforces a sensible minimum and maximum range for the daily loss limit, reverting with a clear error when the provided value falls outside that range.granted the StableSwapLPCollateral TRANSFERER role.
Status	Fixed, now bounds the maxDailyLossUsd to a minimum of $1 and a maximum of $100,000 represented by MIN_DAILY_LOSS_USD and MAX_DAILY_LOSS_USD respectively.

[I07] Inconsistent modifier ordering between Liquidator and BrokerLiquidator	
Contract	Liquidator.sol
Severity	Informational
Description	**Description** The Liquidator and BrokerLiquidator apply the same two modifiers to their BOT entry points but in opposite order. The Liquidator uses nonReentrant onlyRole(BOT) across its

	nine BOT entry points, while the BrokerLiquidator uses onlyRole(BOT) nonReentrant across its seven BOT entry points. JavaScript `external nonReentrant onlyRole(BOT)` **Impact** Both modifiers execute in the before phase and the function body runs only after both pass, so the end state is identical and there is no security or correctness difference. The ordering still has two minor consequences. 1. Placing onlyRole first checks authorization before the reentrancy slot is written, which follows the checks before effects convention. 2. It also makes the failure path for an unauthorized caller cheaper, because the caller reverts at the role read before paying for the reentrancy slot write, whereas with nonReentrant placed first an unauthorized caller pays for the entered slot write before the role check reverts.
Recommendation	Standardize both contracts on the onlyRole(BOT) nonReentrant order used by BrokerLiquidator, so the Liquidator checks the caller's role before mutating the reentrancy slot.
Status	Acknowledged, no fix implemented. This is purely a cosmetic/gas optimization issue so a fix is optional.

[I08] Unused IERC20 import in LiquidationVault	
Contract	LiquidationVault.sol
Severity	Informational
Description	**Description** LiquidationVault imports IERC20 from the OpenZeppelin interfaces but never references it. All token balance reads use token.balanceOf(address(this)) resolved through using SafeTransferLib for address, all transfers use token.safeTransfer, and token decimals are read through IERC20Metadata, so IERC20 is dead. JavaScript `import { IERC20 } from "@openzeppelin/contracts/interfaces/IERC20.sol";` **Impact** The unused import adds noise and a small compilation cost with no functional purpose.

Recommendation	Remove the IERC20 import. Keep the IERC20Metadata import, since it is used by _oracleValue to read decimals().
Status	Fixed, removed the unused IERC20 import.

[I09] Duplicate balance reads in sellToken and sellBNB	
Contract	LiquidationVault.sol
Severity	Informational

Description	**Description**
	sellToken calls tokenIn.balanceOf(address(this)) twice in immediate succession, once inside the require(... >= amountIn) check and once to set beforeTokenIn. sellBNB has the same pattern with address(this).balance read twice, which uses a cheaper opcode but is still redundant. The same anti pattern also exists in the sell functions of Liquidator and BrokerLiquidator.
	
	JavaScript
	`require(tokenIn.balanceOf(address(this)) >= amountIn, ExceedAmount());`
	`uint256 beforeTokenIn = tokenIn.balanceOf(address(this));`
	`require(address(this).balance >= amountIn, ExceedAmount());`
	`uint256 beforeTokenIn = address(this).balance;`
	
	Impact
	Each call is a separate external staticcall that costs roughly 2600 gas when cold, so one of the two is wasted on every sale.

Recommendation	Optionally read the balance once into the before local and derive the require from it
Status	Acknowledged, no fix implemented. This is purely a cosmetic/gas optimization issue so a fix is optional.

[I10] Pack dailyLossDay and dailyLossAccum into a single storage slot	
Contract	LiquidationVault.sol
Severity	Informational
Description	**Description** The loss guard state in LiquidationVault is spread across five separate uint256 sized slots that are all touched on the hot path _guardSwapLoss, which runs on every guarded sale through sellToken and sellBNB. During a single sale the guard reads oracle twice through _oracleValue, reads maxSwapLossBp for the per swap cap, reads maxDailyLossUsd for the daily cap, reads and sometimes writes dailyLossDay, and

reads and writes dailyLossAccum. Because each of these lives in its own slot, the guard performs several distinct SLOADs where a much smaller number would suffice, and the day rollover and accrual writes hit two separate slots.

All five values are far smaller than 256 bits.

- oracle and revenueCollector are addresses of 160 bits.
- maxSwapLossBp is a parts per million value bounded by BPS_DENOM of 1e6, with a default of 50_000 meaning 5%, so it fits in uint32.
- maxDailyLossUsd is an 8 decimal USD figure with a default of 1000e8, which even at extreme values fits in uint128 or uint96.
- dailyLossDay is a day index of block.timestamp / 1 days, currently around 20000, which fits in uint32.
- dailyLossAccum is an 8 decimal USD value bounded by maxDailyLossUsd, which fits in uint128 or uint96.

```javascript
 /// @dev ResilientOracle used by the sell loss guard; peek() returns 8-decimal USD price.
  address public oracle;
  /// @dev Per-swap loss cap in ppm (BPS_DENOM = 1e6). Default 50_000 = 5%.
  uint256 public maxSwapLossBp;
  /// @dev Per-UTC-day cumulative loss cap, 8-decimal USD. Default 1000e8.
  uint256 public maxDailyLossUsd;
  /// @dev UTC day (block.timestamp / 1 days) the accumulator currently tracks.
  uint256 public dailyLossDay;
  /// @dev Cumulative sell loss (8-decimal USD) recorded so far for `dailyLossDay`.
  uint256 public dailyLossAccum;
  /// @dev Optional revenue collector authorized to withdraw* (funds land on itself). 0 disables it.
  address public revenueCollector;
```

Impact

Keeping them all as full width uint256 in separate slots wastes both the read cost on every sale and the write cost on day rollover and accrual.

Recommendation

Consider optimizing the types and pack the values that are accessed together on the hot path so the loss guard loads and stores far fewer distinct slots.

One workable layout groups the frequently read values into one slot and the daily accumulator pair into another, For example, with the following layout a guarded sale touches two slots instead of five

	• address oracle with uint32 maxSwapLossBp and uint32 dailyLossDay in a first slot • uint128 maxDailyLossUsd with uint128 dailyLossAccum in a second slot
Status	Acknowledged, no fix implemented. This is purely a cosmetic/gas optimization issue so a fix is optional.

[I11] String revert reasons instead of custom errors	
Contract	LiquidationVault.sol, Liquidator.sol and BrokerLiquidator
Severity	Informational

Description	**Description**
	The contract uses custom errors for most failures but still uses string revert reasons in several places, including the ZERO_ADDRESS constant used by initialize and multiple setters, and inline strings such as "amountIn zero" in the sell functions.
	JavaScript
	`string internal constant ZERO_ADDRESS = "zero address";`
	`error NotRegistered();`
	`error NotAuthorized();`
	`error NotWhitelisted();`
	`error ExceedAmount();`
	`error NoProfit();`
	`error SwapFailed();`
	`error WhitelistSameStatus();`
	`error OracleZero();`
	`error OracleLoss();`
	`error DailyLossExceeded();`
	`error InvalidParam();`
	Note: This applies for the Liquidator and BrokerLiquidator contracts as well
	
	Impact
	String reasons cost more in both deployment and revert data gas than a 4 byte custom error, and mixing the two styles is inconsistent. The related liquidators also carry inline string reasons such as the pair and spender equality check.

Recommendation	Consider replacing the remaining string reasons with custom errors, for example define error ZeroAddress()
Status	Acknowledged, no fix implemented. This is purely a cosmetic/gas optimization issue so a fix is optional.

[I12] Liquidator.sellBNB emits the requested amount instead of the actual amount consumed	
Contract	Liquidator.sol

Severity	Informational

Description	**Description**
	In Liquidator.sellBNB, the SellToken event is emitted with amountIn rather than actualAmountIn, even though the function already computes actualAmountIn as beforeTokenIn - address(this).balance and validates it with require(actualAmountIn <= amountIn, ExceedAmount()).
	
	JavaScript
	`/// @dev sell BNB.`
	`/// @param pair The address of the pair.`
	`/// @param tokenOut The address of the output token.`
	`/// @param amountIn The BNB amount to sell.`
	`/// @param amountOutMin The minimum amount to receive.`
	`/// @param swapData The swap data passed to low level swap call. Should be obtained from aggregator API like 1inch.`
	`function sellBNB(`
	`address pair,`
	`address tokenOut,`
	`uint256 amountIn,`
	`uint256 amountOutMin,`
	`bytes calldata swapData`
	`) external nonReentrant onlyRole(BOT) {`
	`require(tokenWhitelist[BNB_ADDRESS], NotWhitelisted());`
	`require(tokenWhitelist[tokenOut], NotWhitelisted());`
	`require(pairWhitelist[pair], NotWhitelisted());`
	`require(amountIn > 0, "amountIn zero");`
	
	`require(address(this).balance >= amountIn, ExceedAmount());`
	
	`uint256 beforeTokenIn = address(this).balance;`
	`uint256 beforeTokenOut = tokenOut.balanceOf(address(this));`
	
	`(bool success, ) = pair.call{ value: amountIn }(swapData);`
	`require(success, SwapFailed());`
	
	`uint256 actualAmountIn = beforeTokenIn - address(this).balance;`
	`uint256 actualAmountOut = tokenOut.balanceOf(address(this)) - beforeTokenOut;`
	
	`require(actualAmountIn <= amountIn, ExceedAmount());`
	`require(actualAmountOut >= amountOutMin, NoProfit());`
	
	`emit SellToken(pair, pair, BNB_ADDRESS, tokenOut, amountIn, actualAmountOut);`
	`}`
	
	Impact

	When the swap consumes less native coin than requested, the event overstates the input actually spent, so off chain accounting and monitoring that rely on the event see an incorrect input amount. The sibling BrokerLiquidator.sellBNB already emits actualAmountIn, so the two contracts are inconsistent. This is cosmetic and does not affect on chain state or funds.
Recommendation	Consider emitting the actualAmountIn in place of amountIn in the SellToken event within Liquidator.sellBNB, matching the ERC20 sellToken path and the BrokerLiquidator.sellBNB behavior, so the event reflects the amount actually spent.
Status	Fixed, now emits the actualAmountIn within the SellToken event instead of amountIn.

[I13] Unused IBroker import in BrokerLiquidator	
Contract	BrokerLiquidator.sol
Severity	Informational
Description	**Description** BrokerLiquidator imports both IBroker and IBrokerBase from the broker interfaces, but only IBrokerBase is ever referenced. All broker interactions use IBrokerBase, including the MARKET_ID() read and the liquidate(...) calls across flashLiquidate, liquidate, liquidateSmartCollateral, and flashLiquidateSmartCollateral JavaScript `import { IBroker, IBrokerBase } from "../broker/interfaces/IBroker.sol";` **Impact** IBroker is never used, so it is dead in the import and adds noise with no functional purpose.
Recommendation	Consider removing IBroker from the import so it reads import { IBrokerBase } from "../broker/interfaces/IBroker.sol";
Status	Fixed, now removes the IBroker import from the BrokerLiquidator contract.

[I14] Deployment leaves a single EOA holding every LiquidationVault role with no handover	
Contract	deploy_liquidationVault.sol
Severity	Informational
Description	**Description** deploy_liquidationVault.sol initializes the vault with initialize(deployer, deployer, deployer, deployer), so the deployer EOA simultaneously holds DEFAULT_ADMIN, MANAGER, PAUSER, and BOT.

JavaScript

```
/// @dev Deploys the LiquidationVault (impl + UUPS proxy). Roles are
initialized to the deployer and
///      then handed over via a separate transfer-role script (mirrors
deploy_liquidator flow).
contract LiquidationVaultDeploy is DeployBase {
  function run() public {
    uint256 deployerPrivateKey = _deployerKey();
    address deployer = vm.addr(deployerPrivateKey);
    console.log("Deployer: ", deployer);
    vm.startBroadcast(deployerPrivateKey);

    // Deploy LiquidationVault implementation
    LiquidationVault impl = new LiquidationVault();
    console.log("LiquidationVault implementation: ", address(impl));

    // Deploy LiquidationVault proxy. initialize(admin, manager, pauser,
bot).
    ERC1967Proxy proxy = new ERC1967Proxy(
      address(impl),
      abi.encodeWithSelector(impl.initialize.selector, deployer,
deployer, deployer, deployer)
    );
    console.log("LiquidationVault proxy: ", address(proxy));

    vm.stopBroadcast();
  }
}
```

Impact

The file comment states that roles are handed over through a separate transfer role script, but no such script exists in the deployment directory, and neither deploy_liquidationVault.sol nor deploy_liquidationVaultConfig.sol performs any grantRole, revokeRole, or renounceRole. After both scripts run, a single private keykey can upgrade the contract through DEFAULT_ADMIN, drain the entire pooled reserve through MANAGER withdraw, pause the system, and sell inventory.

Recommendation	Add a role transfer script that runs immediately after deployment, assigning DEFAULT_ADMIN to the TimeLock, MANAGER to the multisig, PAUSER to the operations signer, and BOT to the hot key, then renounces every role held by the deployer.
Status	Fixed, now includes the deploy_liquidationVaultTransferRole.sol script that grants the necessary roles, then renounces every deployer role, with post-condition asserts.

[I15] Reserve migration is not scripted and hardcoded amounts can revert	
Contract	deploy_liquidationVaultConfig.sol
Severity	Informational

Description	**Description**
	The reserve migration from the liquidators into the vault through collectERC20 and collectETH is described in comments but not scripted.
	
	JavaScript
	`/// Reserve migration (vault.collectERC20 / collectETH) must run AFTER setFundSource, since the pull is`
	`/// on-demand and consumes local balance first. Recommended: execute atomically via multisig.`
	
	Impact
	Because reflow also sweeps whole balances into the vault on liquidations, a later collection that uses a hardcoded amount can revert if reflow has already moved those funds. This is an operational hazard rather than a code defect.

Recommendation	If migration is scripted, read live balances at execution time rather than hardcoding amounts, and account for the fact that reflow may have already moved part or all of a liquidator's balance into the vault.
Status	Fixed, the deploy_liquidationVaultWire.sol script now reads the current Liquidator/BrokerLiquidator balances to ensure migration only migrates the current balances in a single atomic transaction, preventing reverts.

[I16] RevenueCollector wiring is only half scripted	
Contract	deploy_liquidationVaultConfig.sol
Severity	Informational

Description	**Description**
	When a revenue collector is configured, step 4 calls vault.setRevenueCollector(rc) but does not perform the reciprocal RevenueCollector.updateLiquidator(vault, true), which is mentioned only in a comment.
	
	JavaScript
	`/// Reserve migration (vault.collectERC20 / collectETH) must run AFTER setFundSource, since the pull is`
	`/// on-demand and consumes local balance first. Recommended: execute atomically via multisig.`
	
	Impact
	Without both sides wired, the collector cannot pull, because its claim path requires the vault to be present in its liquidator set. Given that registering the vault as a collector grants the collector uncapped pull authority over the whole vault balance, this decision should be made and executed deliberately rather than left partial.

Recommendation	Consider scripting both sides of the wiring together, or explicitly document that the collector side is a separate reviewed step, and confirm that enabling the collector is intended given the pull authority it grants over the vault.
Status	Acknowledged, ListaDao has indicated that this is kept config-only by design , along with documenting the reciprocal RevenueCollector.updateLiquidator(vault, true) as a deliberate, separately-reviewed governance step, given registering the vault grants the collector uncapped pull authority.

[I17] V3Liquidator integration considerations	
Contract	V3Liquidator
Severity	Informational
Description	**Description** The following are an non-exhaustive list of considerations for future integration of the LiquidationVault with the V3Liquidator contract: 1. First, redeemV3Shares is onlyRole(BOT) and forwards a caller supplied receiver straight into IV3Provider.redeemShares. A compromised BOT can redeem V3 shares held by the liquidator, which are seized collateral, and send the underlying token0 and token1 to any address it chooses, bypassing the MANAGER only withdraw. Ensure that these funds after redemption are reflowed back to the LiquidationVault 2. Similar Liquidator and BrokerLiquidator as mentioned in [M03], the flash path with redeemShares == true swaps token0 and token1 into loanToken using BOT supplied swap calldata and checks only that d.loanToken.balanceOf(address(this)) < repaidAssets reverts. There is no oracle loss guard and no daily cap, so a BOT can route the swap through a venue it controls and leak the liquidation bonus, the same issue identified for the other liquidators. 3. The pooled inventory model does not fit V3 collateral. The collateral of a V3 market is the V3Provider share token, which is not tradeable on a whitelisted pair and can only be redeemed through the provider, which the vault has no integration for. If the basic liquidate path, which holds shares, reflowed them, the shares would be permanently stranded in the vault. So the V3Liquidator must always redeem shares into token0 and token1 inside the liquidator and reflow the underlying, never the shares. 4. The basic liquidate function passes empty callback data, so Moolah never invokes onMoolahLiquidate. The contract must already hold the loanToken, which it max approves to Moolah with type(uint256).max and then clears to 0. There is therefore no callback in which to pull the exact shortfall from the vault.

	Vault funding of this path would require pulling before the Moolah call, but repaidAssets is not known until Moolah computes it, so the contract would have to over pull and reflow the remainder, or route vault funded liquidations through a callback path instead. 5. the V3Liquidator needs the a. fundSource + setFundSource (with the same "vault must already list me" guard) b. withdraw* relaxed from onlyRole(MANAGER) to MANAGER || msg.sender == fundSource so the vault's collect* can pull c. an else if (fundSource != address(0)) pull branch in onMoolahLiquidate (for the non-redeem flash path) d. _reflow of loanToken, token0, token1, and native after each liquidation/redeem (but not the V3 shares) e. set _setRoleAdmin(BOT, MANAGER) in initialize (no separate setBotRoleAdmin migration needed if it deploys fresh). 6. There are native currency considerations. The redeem path unwraps the wrapped native leg to the native coin and, when the loanToken equals the wrapped native, wraps the entire contract native balance back, which greedily captures any unrelated native the contract holds.
Recommendation	See above description, a separate audit should be scheduled once LiquidationVault support is applied for the V3Liquidator
Status	Acknowledged, OOS of this audit but listed for future considerations.